



UR-CST
GAKO- CAMPUS
CE-LEV 4

REAL TIME SYSTEM

ASSIGNMENT ON 02 OCT 2026

GRP_MEMBERS

: 223026876

: 223026681

: 223026693

Table of Contents

Mini Project 1.1.....	3
Answers to tasks.....	5
Lab task2:.....	6
Lab task 3:.....	6
Lab task4 :.....	6
Mini Project 1.2: Measurement-Based WCET Estimation and Confidence Tool.....	8
comparison table.....	9
Mini Project 1.3: Real-Time Jitter Analyzer Under Simulated System Noise.....	10
Discussion — Mapping disturbances to Section 1.6 sources.....	10
Combined Result table.....	10

Mini Project 1.1

Setup. A simulated single-task workload was timed in VS Code with Python 3.14.7. The job function's runtime scales with an input workload parameter; workloads were drawn uniformly from 10,000 to 200,000 inner-loop iterations. 120 consecutive releases were scheduled with a nominal period $T = 100$ ms and a relative deadline $D = 0.8 \cdot T = 80$ ms.

Execution time. Over 200 isolated calls the mean execution time was 23.5 ms, with a 95th percentile of 40.8 ms and a measured maximum of 52.4 ms. When the same job ran back-to-back inside the periodic release loop, the observed maximum jumped to 101.2 ms and the mean rose to 45.1 ms — a factor of roughly two higher than the isolated case. This gap is exactly the phenomenon Section 1.6 describes (cache effects, OS noise, garbage collection, interpreter overhead) and is the reason Section 1.7 insists that a measured maximum is only a WCET estimate, not a proven bound.

Jitter. Release jitter was tight — standard deviation 0.34 ms, maximum 3.08 ms — confirming that the Windows sleep-based release loop is adequate for coarse period timing. Completion jitter was far larger — standard deviation 23.7 ms, maximum 101.4 ms — because completion inherits the job's variable execution time in addition to the small release jitter. Section 1.3 distinguishes these two quantities for exactly this reason: they have different causes and different magnitudes.

Here is jitter histogram picture:

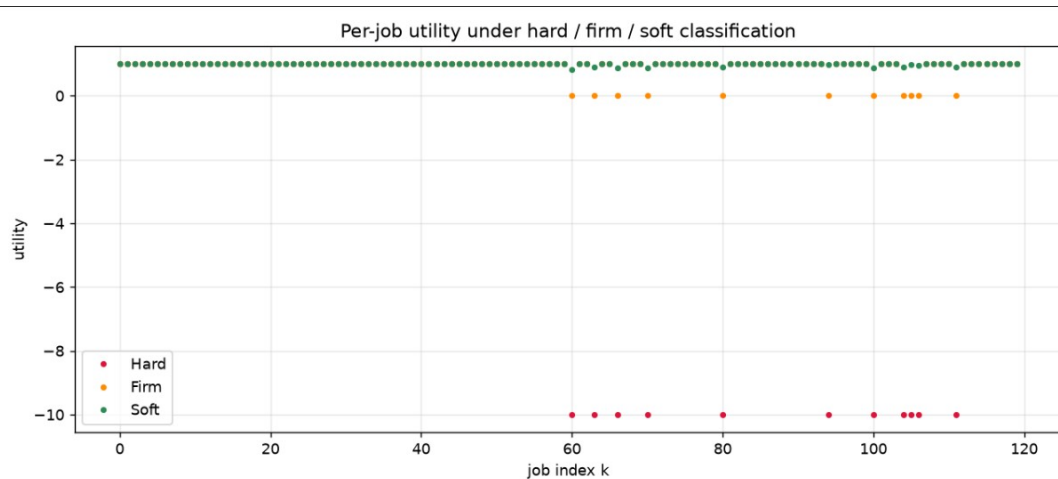


Figure 1. Release jitter and completion jitter histograms for 120 consecutive jobs ($T = 100$ ms, $D = 80$ ms).

Deadline behavior. Of 120 jobs, 109 met the 80 ms deadline and 11 missed it (hit fraction 0.908). The misses occurred at job indices 60, 63, 66, 70, 80, 94, 100, 104, 105, 106, and 111 — not clustered, but scattered, as expected from a workload with a heavy right tail.

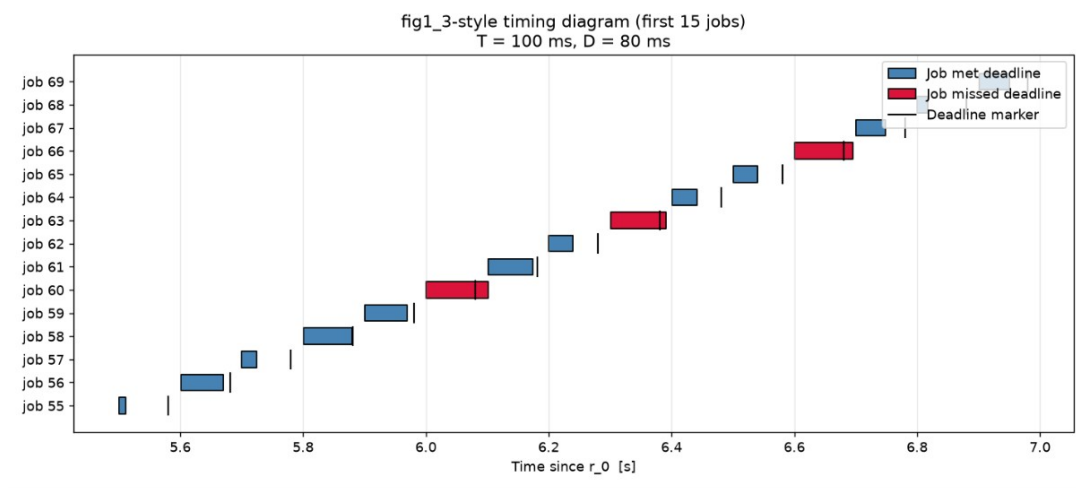


Figure 2: style timing diagram

Utility classification. Applying the fig1_1 cost functions to the identical response-time sequence:

- **Hard mean utility** = -0.008 (pulled negative by 11 catastrophic misses at -10 each)
- **Firm mean utility** = 0.908 (equal to the hit fraction; late results worth exactly zero)
- **Soft mean utility** = 0.991 (misses were only slightly late, so $\exp(-\text{late}/T) \approx 0.9$)



Figure 3: response and utility

The timing behaviour is identical across all three classifications — same 11 misses, same response times. Only the cost of lateness differs. This is the central point of Section 1.2.

This task, as configured, has a firm character. Real deployment domains:

- **Industrial control / automotive ECU (Section 1.2, hard).** A control loop that occasionally finishes ~20 ms late would destabilize the closed-loop plant. Here the same task would be classified hard and would need a validated static WCET (Section 1.7) and offline schedulability analysis (Unit 2). The measured max of 101.2 ms already exceeds the 80 ms deadline, so it would fail the hard requirement as it stands.
- **Video conferencing / streaming (Section 1.2, firm or soft).** A late frame is dropped or gracefully degraded — exactly the observed behaviour: 91% of frames on time, the rest slightly late. Firm or soft would be the correct classification.

- **Interactive UI / best-effort telecom (Section 1.2, soft).** Even a late result is worth something. Soft matches best here, since our missed jobs still carried utility ≈ 0.9 .

Figure 4. Utility of a completed task's result as a function of lateness ($\max(0, R - D)$), comparing hard (immediate collapse), firm (sharp drop to zero), and soft (gradual exponential decay with $\tau = T$). Analog of fig1_1:

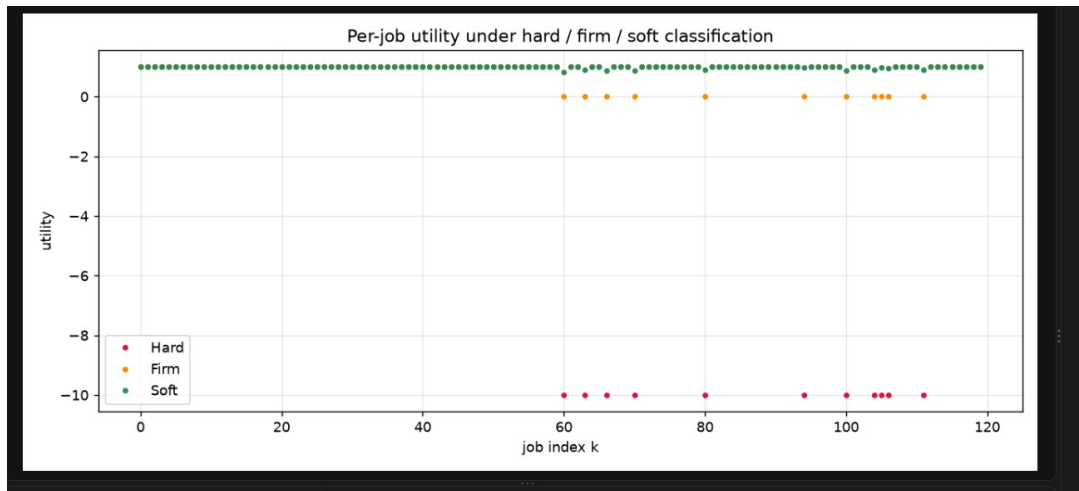
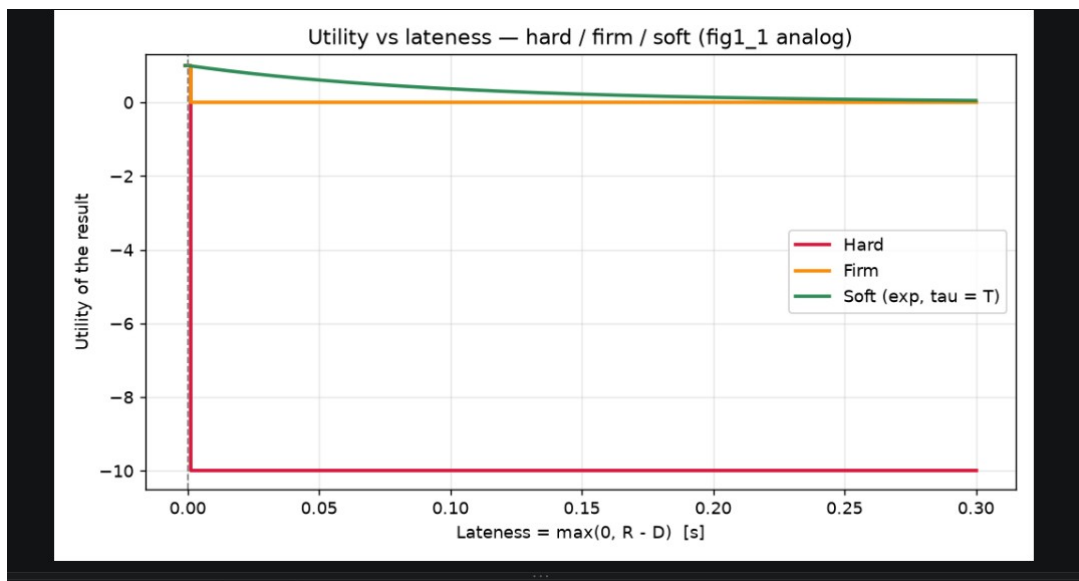


Figure 5. Per-job utility under the three classifications for the same 120 response-time samples. The hard, firm, and soft sequences coincide on on-time jobs and separate only on the 11 misses.



Conclusion. This project turned the theory of Sections 1.2–1.4 into something measurable. From my own data I saw that correctness depends on both value and time, that the mean hides the tail that actually causes misses, that release jitter and completion jitter are distinct phenomena with very different magnitudes, and that the same response-time sequence has completely different engineering consequences depending on whether the deadline is hard, firm, or soft.

Answers to tasks

by adding a background thread that continuously burns CPU while the periodic job runs, imitating the OS noise and co-running-task interference described in Section 1.6. Compared to the unloaded baseline, the mean execution time rose from 45.1 ms to 55.8 ms and the worst case from 101.2 ms to 128.2 ms. More importantly, release jitter increased by a factor of about 17 (std 0.34 ms $\rightarrow$ 6.00 ms; max 3.08 ms $\rightarrow$ 42.19 ms) because the OS now has to arbitrate between two runnable threads. The consequence was a doubled miss count — hit fraction fell from 90.8% to 80.8% — showing that the tail behaviour, not the average, governs real-time correctness, exactly as Section 1.4 claims.

Lab task2:

By re-measuring the job function under three input regimes: ordinary random workloads (10k–200k iterations), adversarial workloads (all 200k), and heavy-tail workloads (all 400k). The maximum observed execution time climbed monotonically — 145 ms ordinary, 164 ms adversarial, 336 ms heavy — confirming that harder inputs reveal longer paths. But repeating the same ordinary campaign on the same code produced a max of 145 ms versus the 52 ms I measured in Step 2, a factor-of-two difference caused entirely by the machine's changing state. This proves Section 1.7's point: measured WCET is never a sound upper bound, because no finite test campaign can guarantee the true worst case has been exercised. Only static analysis over the code's control-flow graph can give a certified WCET for a hard real-time task.

Referring to "lt2_data.npz" r .

Lab task 3:

Holding the same 120 response times fixed, I compared three soft-utility decay rates: fast ($\tau = 0.05 \cdot T$), baseline ($\tau = T$), and slow ($\tau = 5 \cdot T$). On the 11 missed jobs, mean utility ranged from 0.23 (fast) to 0.98 (slow) — a factor of four — while the mean utility across all 120 jobs changed only slightly ($0.93 \rightarrow 1.00$), because only 11 jobs were affected. This shows the decay rate is a design decision that encodes how much lateness the application can tolerate. A video frame delivered one full period late is visually useless (fast decay); a UI response 200 ms late is still perceptible (slow decay). The value of τ must therefore be justified by the application requirement, not chosen arbitrarily.

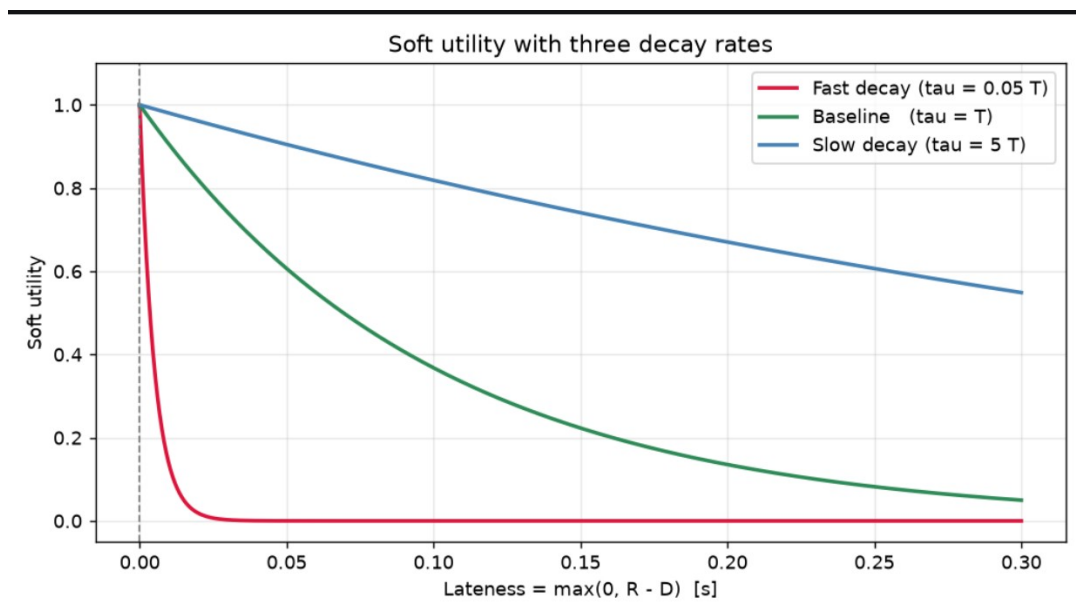


Figure 6. soft utility with three decay.

Lab task4 :

by running the periodic loop with $T = 50, 100, 150$, and 200 ms, keeping $D = 0.8 \cdot T$. The results split into two clear regimes. For $T = 50$ and 100 ms, jitter was huge (std 22.5 and 12.1 ms; as a fraction of T , 45% and 12%) because the mean execution time (~ 55 ms) approached or exceeded the period — the release loop could not start the next job on time and lateness accumulated from one job to the next. For $T = 150$ and 200 ms, jitter collapsed to sub-millisecond ($< 1\%$ of T) and the hit fraction rose to 96% and 100%. This is the clearest demonstration of Section 1.3's point: jitter must be judged relative to the period, not in absolute terms. The same few tens of milliseconds of lateness is catastrophic at $T = 50$ ms but negligible at $T = 200$ ms, and once execution times approach the period, jitter compounds rather than stays bounded.

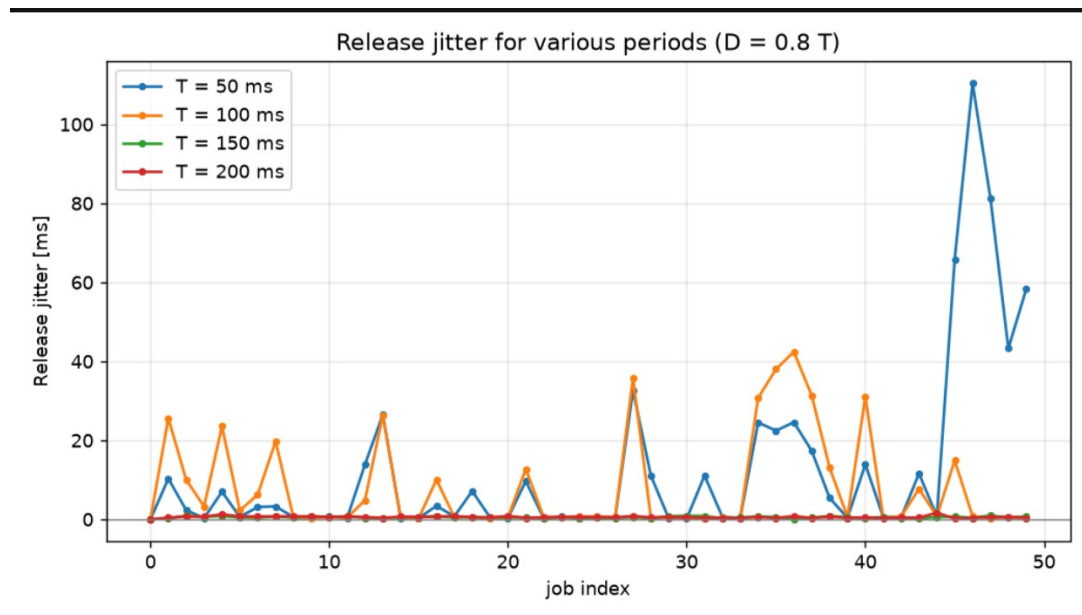


Figure: 7.lt4_jitter_vs_T.png and lt4_jitter_fraction.png.

Mini Project 1.2: Measurement-Based WCET Estimation and Confidence Tool

Soundness and pessimism in measurement-based WCET. Section 1.7 distinguishes two families of WCET estimation. Static analysis reconstructs the control-flow graph of a program, bounds every loop and recursion, and models the target hardware to produce an execution-time bound that is sound — a guaranteed upper bound — but often pessimistic because it must consider hardware states and path combinations that rarely co-occur. Measurement-based estimation, the family implemented by this tool, is the opposite: cheap, easy to apply, and close to observed reality, but unsound because no finite test campaign can guarantee it exercised the true worst case.

The results of this project illustrate both halves of that trade-off directly. For `early_exit_scan`, the random strategy produced a maximum of 24.1 μs while the adversarial strategy — a full scan of a 500-element list with the target absent — produced 106.7 μs , a 4.4 $\times$ difference. Both strategies were run on the same function; the only difference was the distribution of inputs. Had the tool been run with the random strategy alone, it would have reported a WCET estimate roughly four times smaller than the true worst case. This is the empirical demonstration of why measurement-based WCET cannot be certified: the estimate is a property of the input campaign, not of the code.

For `nested_conditional`, the same effect appeared but milder: the random strategy (uniform x over $[-10, 1999]$) hit the deepest branch only occasionally, producing a maximum of 181.0 μs , while the adversarial strategy ($x = 1200$, which satisfies both $x \geq 1000$ and $x \% 6 == 0$) produced 220.7 μs , a 1.22 $\times$ difference. The random strategy did occasionally reach the heavy branch, but not reliably, and never with the slowest sub-branch consistently favoured.

Where the tool was confident, and where it underestimated. The tool estimated `data_dependent_loop` most confidently. Because the random range ($n \in [50, 500]$) already includes the adversarial value $n = 500$ many times, the bulk of both distributions — down to the 99th percentile — was nearly identical (39.7 μs random vs 38.3 μs adversarial). For this function, the adversarial strategy is essentially a subset of the random one, so the random run alone would already have captured a near-worst-case sample.

Paradoxically, the random run produced a larger maximum (81.2 μs) than the adversarial run (46.5 μs). That outlier cannot be explained by control flow — the two strategies execute identical code on identical inputs at their slowest — and must therefore reflect an environmental event outside the function: a garbage-collection pause, an OS preemption, a cache eviction, or interpreter overhead. This is a direct demonstration of Section 1.6's claim that modern platforms introduce non-determinism that is independent of the program's logic. It also shows that a single measured maximum is not a stable property of the function at all, but of the function and the machine state at the time of measurement.

By contrast, `early_exit_scan` and `nested_conditional` were likely underestimated by the random strategy alone, and only the adversarial strategy exposed the true cost of their longest paths. This matches the brief's expectation: the tool's honesty depends on the input campaign, and the tool makes no claim that its random-strategy result alone is a sound WCET.

Margined recommendations. Applying a 40% safety margin — consistent with the 60–70% utilisation-budget convention of Section 1.7 — the tool produces the following recommended WCET-for-scheduling values:

Function	Working measured max (μs)	Margined recommendation (μs)
<code>early_exit_scan</code>	106.7	149.4
<code>data_dependent_loop</code>	81.2	113.7
<code>nested_conditional</code>	220.7	309.0

These margined values are the number a scheduling analysis (Unit 2) would use, but they remain estimates, not proofs.

Why certification would not accept this tool's output. A certification authority operating under IEC 61508, ISO 26262, DO-178C, or IEC 62304 (Section 1.8) requires objective, documented evidence that a safety-relevant task will meet its deadline under the worst case the hardware and software can produce. Three properties of this tool prevent it from satisfying that requirement. First, its output is not sound: the tool makes no guarantee that the true worst-case path was exercised, and the `early_exit_scan` result shows this gap can be a factor of four. Second, its output is not stable: rerunning the same campaign on the same code may produce a different maximum, as the `data_dependent_loop` outlier demonstrates. Third, its output is not traceable to the code's control-flow graph: the tool treats the function as a black box, so there is no way to argue that a particular structural path was covered by measurement.

A certifiable WCET for a hard real-time task therefore requires static analysis over the program's control-flow graph, combined where appropriate with measurement-based segment timing (the "hybrid" approach of Section 1.7) and a documented justification of the margin. This tool, by design, produces a defensible estimate together with an explicit statement of its limitations — exactly the role the brief asks of it. Its value is not that it produces a certified number, but that it produces a number with a clear, honest account of what it does and does not guarantee.

comparison table.

Table . Random vs adversarial vs margined WCET estimate per target function (N = 200 samples per strategy; 40% safety margin per Section 1.7's 60–70% utilisation-budget convention).

Function	Mean (μs)	Std (μs)	Min (μs)	Max (μs)	p90 (μs)	p95 (μs)	p99 (μs)	Max / p99
<code>early_exit_scan</code>	7.2	1.2	5.5	12.5	7.6	9.4	11.8	1.06
<code>data_dependent_loop</code>	11.7	6.0	2.4	22.6	20.1	21.1	22.0	1.03
<code>nested_conditional</code>	27.9	33.0	0.4	161.5	99.0	100.6	108.4	1.49

Mini Project 1.3: Real-Time Jitter Analyzer Under Simulated System Noise

Discussion — Mapping disturbances to Section 1.6 sources

Section 1.6 identifies six concrete sources of non-determinism in modern platforms: cache effects, interrupts, DMA, pipeline hazards, OS jitter, and multicore interference. The three disturbances implemented in this project each approximate one or two of those sources, and the correspondence — and its limits — is worth stating explicitly.

Combined Result table

Mechanism	Setting	Release jitter std (ms)	Completion jitter std (ms)	Response time max (ms)	Hit fraction
1. Background CPU load	intensity = 0.00	1.456	1.450	16.187	1.0000
	intensity = 0.25	0.739	0.749	3.949	1.0000
	intensity = 0.50	1.686	1.683	6.157	1.0000
	intensity = 0.75	2.504	2.511	8.493	1.0000
	intensity = 1.00	4.699	4.700	22.282	1.0000
2. Extra delay	p = 0.05, d = 5 ms	1.092	1.636	14.475	1.0000
	p = 0.10, d = 5 ms	0.205	1.821	7.758	1.0000
	p = 0.20, d = 5 ms	0.461	2.227	8.123	1.0000
	p = 0.50, d = 5 ms	0.593	2.770	7.964	1.0000
	d = 1 ms, p = 0.10	1.524	1.603	17.456	1.0000
	d = 5 ms, p = 0.10	2.061	2.702	28.674	1.0000
	d = 10 ms, p = 0.10	0.433	3.494	12.088	1.0000
	d = 20 ms, p = 0.10	0.948	7.056	32.023	1.0000
3. Exec-time variability	normal, spread = 0	0.580	0.578	1.063	1.0000
	normal, spread = 1000	1.904	1.897	2.547	1.0000
	normal, spread = 3000	0.807	0.855	2.315	1.0000
	normal, spread = 6000	0.729	1.004	3.360	1.0000
	normal, spread = 10000	0.408	0.989	5.176	1.0000
	heavy-tail, mean = 2000	1.759	1.781	3.722	1.0000
	heavy-tail, mean = 4000	0.446	0.795	5.605	1.0000
	heavy-tail, mean = 6000	0.290	0.956	9.521	1.0000
	heavy-tail, mean = 8000	0.428	1.814	16.576	1.0000

Background CPU load ↔ OS jitter and multicore interference. The background thread competes for CPU time, forcing the OS scheduler to arbitrate between two runnable threads. This is the closest analogue in the tool to the general "OS noise" category of Section 1.6: scheduling-tick handling, background daemons, and deferred interrupt processing all steal processor time unpredictably. It also stands in for multicore interference, though imperfectly — in this simulation both threads share a single logical core through the OS scheduler, whereas on a real multicore system they would run truly in parallel while still contending for shared caches, memory bandwidth, and interconnect. The analogy therefore captures the scheduling aspect of multicore interference but not the hardware contention aspect.

Bernoulli random extra delay ↔ interrupts and DMA stalls. The extra-delay hook fires at random release instants and inserts a fixed-magnitude sleep before the job body. This models any rare event that delays the start of a task: an interrupt handler executing at a priority above the task, a DMA transfer stalling processor memory accesses, or a deferred workqueue item running at the wrong moment. The analogy is quite good for interrupt-induced delay: both cases produce occasional, bounded, externally triggered lateness. Where it breaks down is determinism of the magnitude: real interrupts have variable handler duration that depends on what the handler has to do, whereas our model inserts a constant-magnitude delay. A variable-magnitude version would be a more faithful analogue of interrupt latency under varying handler work.

Per-job execution-time variability ↔ cache effects and pipeline hazards. The job body's workload size is drawn from a distribution, so two calls to the same code take different amounts of time. This is the most faithful simulation of cache- and pipeline-related non-determinism, because the mechanism by which real cache behaviour causes variable execution time is precisely that the same instructions take a different number of cycles depending on whether the data they touch was already in cache. A heavy-tailed workload distribution captures the fact that most calls hit the fast path, but a small fraction miss and pay a large penalty — exactly the shape of cache-miss-induced latency distributions. The analogy breaks down in one respect: real cache effects depend on the history of accesses across tasks, so interference from other tasks can evict a task's data mid-execution, a preemptive effect our simulation does not model.

Common thread — jitter without misses. Across all three disturbances, the same pattern held. Completion-jitter std grew up to 6× (background load) and response-time max grew up to 16× (heavy-tailed execution) while the deadline-hit fraction stayed at 1.000 in every single configuration tested. No configuration in this project produced a single miss, yet every configuration produced substantially more jitter than the baseline. This is the empirical demonstration of the brief's central claim: meeting every deadline and having low jitter are distinct properties. A control engineer relying on hit-rate alone would conclude that the system behaved identically across all configurations; a jitter measurement shows they are profoundly different.

Why this matters for real-time design. Many control algorithms — from PID loops to state estimators to Kalman filters — are derived under the assumption that the sampling interval is constant. When release or completion jitter is significant, the actual sampling interval deviates from the nominal T , and the discrete-time model the controller was designed against no longer holds exactly. The control loop may still be stable (because deadlines are still met) but its performance — overshoot, settling time, steady-state error — will differ from the design specification, and at sufficiently high jitter the loop may become unstable. This is why Section 1.3 stresses that jitter is harmful even when deadlines are individually met, and why a real-time engineering process must specify jitter bounds in addition to deadline bounds.